Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab4

The objectives of the lab

This lab illustrates the fourth lecture (*Unsupervised Learning, Generative Learning and More*). Its goals are:

- to see the *curse of dimensionality* numerically (PC4),
- to compute a principal component analysis and to understand it as a reconstruction method,
- to compare nonlinear dimension reduction methods (MDS, Isomap, t-SNE, UMAP) on the digits and on a Swiss roll,
- to compare clustering methods (k-means, Gaussian mixtures, DBSCAN, agglomerative clustering) and to choose the number of groups,
- to generate new digits with a Gaussian mixture, and to train an autoencoder with `{torch}`.

The optional *To go further* part, after the core, can be skipped at first: it covers the rest of the lecture, in particular the generative models (VAE, flows, diffusion and flow matching, GAN).

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (The curse of dimensionality)

PC4 shows that, for n points uniform in $[0,1]^d$, the norms and the pairwise distances concentrate: $\|X\|/\sqrt{d} \to \sqrt{1/3}$ and $\|X - X'\|/\sqrt{d} \to \sqrt{1/6}$, and the ratio between the largest and the smallest distances tends to 1 as long as $\log(n)/d \to 0$.

1. *Predict before you run:* for 250 points uniform in $[0,1]^d$, what happens to the ratio between the largest and the smallest pairwise distances when d grows? Compute, for $d \in \{2, 10, 100, 500, 1000\}$, the minimum, the maximum and the ratio of the distances to the origin and of the pairwise distances (with `dist()`), divided by $\sqrt{d}$.

2. In $[0,1]^d$, a cube that contains a fraction r of the data (uniform) has a side $r^{1/d}$. Plot this side for $r = 1\%$ and $r = 10\%$ as a function of $d \in [1, 100]$. What does it mean for a "local" method such as the k-nearest neighbors?

3. Random projections do not suffer from the dimension: by the Johnson–Lindenstrauss lemma, projecting n points on $k = O(\log(n)/\varepsilon^2)$ random directions preserves all their pairwise distances up to a factor $1 \pm \varepsilon$. Project 250 Gaussian points of dimension 1000 on $k \in \{10, 50, 200\}$ random Gaussian directions (scaled by $1/\sqrt{k}$), and plot the distribution of the ratios between the projected and the original pairwise distances.

Exercise 2 (Principal component analysis of the digits)

The digits are 8×8 grayscale images (values from 0 to 16): 1797 observations in dimension 64. A PCA looks for the linear subspace of dimension q that keeps the largest inertia, or equivalently that reconstructs the data with the smallest error (PC4); it is computed by a singular value decomposition of the centered data matrix.

1. Compute the PCA of the digits with `prcomp` (without scaling), and check that its scores are the left singular vectors of the centered matrix multiplied by the singular values (`svd`), and that classical multidimensional scaling (`cmdscale`) of the Euclidean distances gives the same coordinates (up to signs). Plot the proportion of explained variance and the first two scores colored by the digit.

2. Plot the mean digit and the first 6 principal directions as 8×8 images ("eigen-digits"). Then plot the reconstruction of the first 5 digits with $q \in \{2, 5, 10, 20\}$ components, and the mean reconstruction error as a function of q. Relate this error to the explained variance.

Exercise 3 (Nonlinear dimension reduction)

A PCA can only find linear structures. On a *Swiss roll*, a 2D sheet rolled in 3D, the interesting coordinate is the position along the roll, which is not linear. *Isomap* replaces the Euclidean distances by *geodesic* distances (shortest paths in a neighborhood graph) and applies MDS to them; *t-SNE* and *UMAP* preserve the neighborhoods rather than the distances.

1. Implement Isomap: build the graph of the 10 nearest neighbors (`dbscan::kNN`), compute the shortest path distances (`igraph::distances`) and apply `cmdscale`. Compare the PCA and the Isomap embeddings of the Swiss roll, colored by the position along the roll `t_roll`.

   To compare embeddings, we use the proportion of the 10 nearest neighbors of each point (in the original space) that are still among its 10 nearest neighbors in the embedding.

2. Embed the digits in 2D with PCA, Isomap, t-SNE (`Rtsne`, on the first 50 principal components, perplexity 30) and UMAP (`uwot::umap`). Plot the embeddings colored by the digit, and compare their neighbor preservation. Then try the perplexities 5 and 100 for t-SNE.

Exercise 4 (Clustering)

We compare the clustering methods of the lecture on three toy datasets with known groups, and on the digits. The agreement between a clustering and the true groups is measured with the *adjusted Rand index* (ARI): the proportion of pairs of points on which the two partitions agree (both in the same group, or both in different groups), corrected so that a random partition gets 0 on average and identical partitions get 1 (`mclust::adjustedRandIndex`).

1. On each toy dataset, run k-means (`nstart = 20`), a Gaussian mixture (`Mclust`, with the true number of groups), DBSCAN (`dbscan`, $\varepsilon = 0.5$, `minPts = 10`; its noise points form a group of their own) and agglomerative clustering with the single, average and Ward linkages (`hclust` + `cutree`, true number of groups). Report the ARI of each method. Which assumptions explain the results?

2. On the first 10 principal components of the digits, choose the number of groups $K \in \{2, \dots, 15\}$ with the *elbow* of the within-cluster inertia of k-means, the average *silhouette* of k-means, and the BIC of Gaussian mixtures with diagonal covariances (`Mclust(..., modelNames = "VVI")`). Compare the chosen numbers with the 10 digits, and the ARI of the clusterings with the labels.

Exercise 5 (A first generative model: Gaussian mixtures)

A density model can be used to *generate* new data: a Gaussian mixture is sampled by drawing a component, then a Gaussian vector. In dimension 64, the covariances of a mixture are hard to estimate, so we fit the mixture on the first principal components and map the samples back to the pixels.

- Fit a Gaussian mixture with 12 components and full covariances (`Mclust(..., G = 12, modelNames = "VVV")`) on the first 15 principal components of the digits, draw 30 new points with `mclust::sim`, map them back to the pixel space and plot them. Compare with a mixture on 5 components only.

Exercise 6 (Autoencoders with 'torch')

An autoencoder is a neural network trained to reconstruct its input through a low-dimensional *code*: an encoder $x \mapsto z$ and a decoder $z \mapsto \hat{x}$, trained to minimize $\|x - \hat{x}\|^2$. With linear layers and the squared loss, the best autoencoder spans the same subspace as the PCA; nonlinear layers can find curved structures.

1. Train a *linear* autoencoder $64 \to 2 \to 64$ (`nn_linear` layers, full batch, Adam, 1500 epochs), and compare its reconstruction error with the PCA with 2 components. Check that the subspace spanned

by the decoder weights is the one of the first two principal directions, by computing the cosines of the *principal angles* between the two subspaces (the singular values of $Q_1^\top Q_2$, where Q_1 and Q_2 are orthonormal bases).

2. Train a deep autoencoder $64 \to 128 \to 32 \to 2 \to 32 \to 128 \to 64$ with `tanh` activations. Compare its reconstruction error with the linear one, and plot the 2D codes of both, colored by the digit.

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: $\star$ direct extension, $\star\star$ new method, $\star\star\star$ longer or research-flavored.

Block	Lecture	Stars
Semi-supervised learning: label propagation	More than supervised or unsupervised	$\star\star$
Pairwise comparisons: the Bradley–Terry model	Comparison learning	$\star\star$
Structured data: blocked cross validation	Structured learning	$\star$
Classification metrics: ROC and precision-recall	Metrics	$\star$
Multiple correspondence analysis	Dimension reduction (MFA/MCA)	$\star\star$
ICA and NMF	Dimension reduction	$\star\star$
Kernel PCA and Laplacian eigenmaps	Dimension reduction	$\star\star$
Stability and parameters of t-SNE and UMAP	Comparing methods	$\star\star$
k-medoids and HDBSCAN	Clustering	$\star$
Spectral clustering	Clustering	$\star\star$
EM for Gaussian mixtures	Model-based clustering (PC4)	$\star\star$
Two moons, a toy generative problem	Generative modeling	$\star$
A variational autoencoder	Latent variables, ELBO	$\star\star$
Langevin sampling	Energy-based models, score	$\star\star$
A normalizing flow: Real NVP	Flows	$\star\star$
Flow matching	Diffusion and flow matching	$\star\star$
Guidance	Diffusion and flow matching	$\star\star$
A generative adversarial network	GAN	$\star\star$
An autoregressive model of the digits	Factorization, sequences	$\star\star$
Comparing generative models	Generative modeling	$\star\star\star$

Exercise 7 (**Semi-supervised learning: label propagation** $\star\star$)
When only a few labels are known, the unlabeled points can still help: *label propagation* spreads the known labels along a neighborhood graph, assuming that close points share their label.

- Keep the labels of 20 random digits only (2 per class). Propagate them on the graph of the 10 nearest neighbors (in the space of the first 30 principal components): iterate $F \leftarrow \alpha S F + (1-\alpha) Y$, where S is the symmetrically normalized adjacency matrix, Y the one-hot matrix of the known labels and $\alpha = 0.9$. Compare the accuracy on the unlabeled digits with a 1-NN classifier trained on the 20 labeled digits.

Exercise 8 (**Pairwise comparisons: the Bradley–Terry model** $\star\star$)
In *comparison learning*, the data are comparisons ("i beats j") rather than labels. The Bradley–Terry model

assumes $\mathbb{P}(i \succ j) = \sigma(\theta_i - \theta_j)$: it is a logistic regression whose features are $+1$ for i, -1 for j and 0 elsewhere. It is used to rank players, products, or the answers of language models (RLHF, Lecture 6).

- Simulate 10 players with strengths θ drawn from $\mathcal{N}(0, 1)$ and 500 games, estimate the strengths with `glm(..., family = binomial)` (one player fixed at 0), and compare the estimated ranking with the true one and with the ranking by the proportion of wins: first with random opponents, then when each player mostly meets opponents of similar strength (as in sports leagues: the 1 or 2 players just above or below in the true ranking).

Exercise 9 (**Structured data: blocked cross validation** $\star$)
When the observations are not independent (time series, spatial data, repeated measures of the same individuals), a random cross validation puts dependent observations in the training and the validation parts and is optimistic. *Blocked* cross validation keeps the dependent observations together (Lab 1 did it for time series).

- Simulate 50 patients with 20 measures each, \$y = \$ patient effect $+\varepsilon$, and x = noisy copy of the patient identifier effect (a feature which identifies the patient but does not generalize to new patients). Compare the RMSE of a random forest (`ranger`) estimated by a random 5-fold cross validation and by a 5-fold cross validation grouped by patient, with its RMSE on new patients.

Exercise 10 (**Classification metrics: ROC and precision-recall** $\star$)
The *Metrics* section of the lecture: a classifier with scores can be evaluated by its ROC curve (true positive rate against false positive rate, for all the thresholds) and its precision-recall curve. The area under the ROC curve (AUC) is the probability that a random positive gets a higher score than a random negative.

- Train a logistic regression (`glm`) to recognize the digit 8 against the others on half of the digits, plot its ROC and precision-recall curves on the other half (`yardstick`), and check that its AUC is the proportion of (positive, negative) pairs correctly ordered.

Exercise 11 (**Multiple correspondence analysis** $\star\star$)
For categorical data, the analogue of the PCA is the *multiple correspondence analysis* (MCA, `FactoMineR`): it decomposes the table of indicators of the categories, with weights that make it a χ^2 analysis. The lecture uses the `poison` data: 55 children, their symptoms and what they ate.

- Run an MCA on the symptoms and food variables of `poison` (`MCA(poison[, 5:15])`), plot the categories on the first plane, and interpret the first axis.

Exercise 12 (**ICA and NMF** $\star\star$)
Other linear decompositions impose other constraints than orthogonality: *independent component analysis* (ICA) looks for statistically independent (non-Gaussian) components, and *non-negative matrix factorization* (NMF) for a decomposition $X \approx WH$ with $W, H \geq 0$, whose components are *parts* of the images.

- Compute 10 components of the digits with `fastICA` and with an NMF implemented with the multiplicative updates of Lee and Seung ($H \leftarrow H \odot \frac{W^\top X}{W^\top W H}$, $W \leftarrow W \odot \frac{X H^\top}{W H H^\top}$, 200 iterations), and plot them as images. Compare with the principal directions.

Exercise 13 (**Kernel PCA and Laplacian eigenmaps** $\star\star$)
Kernel PCA (PC4) performs a PCA in the feature space of a kernel: it only needs the centered kernel matrix HKH and its eigenvectors. *Laplacian eigenmaps* embed the points with the eigenvectors of the smallest nonzero eigenvalues of the Laplacian $L = D - W$ of a neighborhood graph (the same graph as Isomap), which keep the neighbors close.

- Compute a Gaussian kernel PCA of the Swiss roll (by hand: kernel matrix, double centering, eigenvectors scaled by $\sqrt{\lambda}$) and its Laplacian eigenmap (graph of the 10 nearest neighbors, normalized Laplacian). Compare with the PCA and Isomap embeddings of the core part.

Exercise 14 (Stability and parameters of t-SNE and UMAP ⋆⋆)
t-SNE and UMAP depend on random initializations and on parameters that change the maps a lot. We measure each map with two scores: the *local* score is the neighbor preservation of the core part; the *global* score is the Spearman correlation between the distances of the 10 digit centroids in the original space and in the map (does the map keep which digits are far apart?).

1. Run t-SNE (perplexity 30) with 3 seeds, first with its default random initialization, then initialized with the first two principal components (scaled to a small standard deviation, argument `Y_init`), and UMAP with 3 seeds. Plot the maps and compute their two scores. What is stable?
2. Run UMAP (seed 1) with `n_neighbors` in $\{5, 15, 50, 200\}$ and `min_dist` in $\{0, 0.1, 0.5, 0.99\}$. Plot the 16 maps, and the two scores as functions of the parameters.
3. Run t-SNE (seed 1) with the perplexities 2, 5, 30 and 100 and, for the perplexity 30, with the early exaggeration factors 4, 12 (the default) and 30 (`exaggeration_factor`). Plot the maps and compute their scores.

Exercise 15 (k-medoids and HDBSCAN ⋆)
k-medoids (PAM) uses actual observations as centers and any dissimilarity, which makes it more robust than k-means. HDBSCAN builds the hierarchy of the DBSCAN clusterings for all the values of ε and keeps the most stable clusters: it needs only a minimal cluster size.

- Compare on the toy datasets the ARI of PAM (`cluster::pam`) and HDBSCAN (`dbscan::hdbscan`, `minPts = 10`) with the methods of the core part.

Exercise 16 (Spectral clustering ⋆⋆)
Spectral clustering embeds the points with the first eigenvectors of the normalized Laplacian of a neighborhood graph (as the Laplacian eigenmaps), then runs k-means in this embedding: it finds groups that are *connected*, not necessarily compact.

- Implement spectral clustering (graph of the 10 nearest neighbors, normalized Laplacian, K eigenvectors of the smallest eigenvalues, rows normalized, k-means) and compare its ARI on the rings with k-means.

Exercise 17 (EM for Gaussian mixtures ⋆⋆)
PC4 shows that the EM algorithm is a coordinate ascent of the ELBO: the E step computes the *responsibilities* $r_{ik} = \pi_k \varphi_k(x_i) / \sum_l \pi_l \varphi_l(x_i)$, the M step updates the proportions, means and covariances with these weights.

- Implement EM for a mixture of 3 Gaussians with full covariances, run it on the blobs, plot the log-likelihood along the iterations (it must never decrease), and compare the result with `Mclust`.

Exercise 18 (Two moons, a toy generative problem ⋆)
The generative blocks below use the *two moons* of the lecture, a 2D distribution which is easy to plot but not Gaussian. A generator is good if its samples look like the data, cover both moons, and do not simply copy the training points.

- *Predict before you run:* what do the samples of a single Gaussian fitted to the two moons look like? And of a mixture of 10 Gaussians? Check it.

Exercise 19 (A variational autoencoder ⋆⋆)
A VAE (PC4, *EM and the ELBO*) has a latent variable $z \sim \mathcal{N}(0, I)$ and a decoder $x|z \sim \mathcal{N}(g_\theta(z), \sigma^2 I)$. As the posterior of z is intractable, an *encoder* $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}(s_\phi^2(x)))$ is trained jointly to maximize the ELBO $\mathbb{E}_{q_\phi}[\log p_\theta(x|z)] - \mathrm{KL}(q_\phi(\cdot|x), \mathcal{N}(0, I))$, with the *reparametrization* $z = \mu_\phi(x) + s_\phi(x)\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, so that the gradient goes through the sampling.

- Train a VAE with a 2D latent space on the two moons ($\sigma = 0.1$, 3000 steps of Adam on mini-batches of 256), and sample new points by decoding $z \sim \mathcal{N}(0, I)$. Then train one on the digits (latent dimension 2, pixels scaled to $[0, 1]$) and decode a grid of codes.

Exercise 20 (**Langevin sampling** $\star\star$)

If the *score* $\nabla_x \log p(x)$ of a density is known, the Langevin dynamics $x_{t+1} = x_t + \frac{\eta}{2}\nabla \log p(x_t) + \sqrt{\eta}\,\xi_t$ ($\xi_t \sim \mathcal{N}(0, I)$) samples from p for small steps and long runs. Score-based models learn the score; here we use the exact score of the mixture of 10 Gaussians fitted above. With isolated modes, the dynamics hardly jumps between them: *annealing* (starting with a smoothed density) helps.

- Implement the Langevin dynamics with the score of the Gaussian mixture `gmm10`, starting from 2000 points uniform in $[-2.5, 2.5]^2$, with $\eta = 0.01$ and 500 steps. Then start all the points from the same point $(-1.2, 0.5)$ (on the left moon): do the samples reach the right moon?

Exercise 21 (**A normalizing flow: Real NVP** $\star\star$)

A normalizing flow is an invertible map $x = G(\omega)$ from a Gaussian ω; its density is given by the change of variables formula, so that it can be trained by maximum likelihood. Real NVP composes *affine coupling layers*: half of the coordinates are kept, the other half is transformed by $x_2 = \omega_2 \odot e^{s(\omega_1)} + t(\omega_1)$, whose Jacobian is triangular with log-determinant $\sum s(\omega_1)$.

- Train a Real NVP with 4 coupling layers (alternating the coordinate that is transformed) on the two moons by maximum likelihood (2000 steps of Adam, mini-batches of 512 points), and sample from it.

Exercise 22 (**Flow matching** $\star\star$)

Diffusion models and flow matching transport a Gaussian to the data distribution along a path of distributions. In (rectified) *flow matching*, $x_t = (1 - t)\omega + tx$ interpolates between a Gaussian sample ω and a data point x, and a network $v_\theta(x_t, t)$ is trained to predict the velocity $x - \omega$ by least squares. New samples are then obtained by solving the ODE $\dot{x}_t = v_\theta(x_t, t)$ from $t = 0$ to 1.

- Train a flow matching velocity field on the two moons (6000 steps of Adam with mini-batches of 1024 points; the time t is given to the network with a few Fourier features $\sin(2\pi k t), \cos(2\pi k t)$), and sample with the Euler scheme using 2, 10 and 100 steps. How many steps are needed?

Exercise 23 (**Guidance** $\star\star$)

A conditional generator $v_\theta(x_t, t, y)$ is trained with the label y of each point (here the moon, ± 1), randomly replaced by a "no label" value (0) 20% of the time. *Classifier-free guidance* then extrapolates from the unconditional to the conditional velocity, $v = v_{\text{free}} + (1 + w)(v_{\text{cond}} - v_{\text{free}})$: a large w gives samples that are more typical of their class, at the price of diversity.

- Train a conditional flow matching on the two moons with their labels (with the functions of the previous block), and sample the upper moon ($y = 1$) with the guidance weights $w \in \{0, 1, 3\}$.

Exercise 24 (**A generative adversarial network** $\star\star$)

A GAN trains a generator $G(\omega)$ against a discriminator $D(x)$ that tries to tell the data from the samples; at the optimum of the discriminator, the generator minimizes the Jensen–Shannon divergence. The training is a minimax game, often unstable, and prone to *mode collapse*: the generator may cover only a part of the distribution.

- Train a GAN on the two moons (4000 steps, generator and discriminator with two hidden layers, non-saturating loss) and plot its samples at several stages of the training. Does it cover both moons?

Exercise 25 (**An autoregressive model of the digits** $\star\star$)

An autoregressive model factorizes the density as $p(x) = \prod_j p(x_j | x_1, \ldots, x_{j-1})$: each conditional is a supervised problem, and sampling generates the coordinates one at a time (the principle of the language models, Lecture 6, applied to pixels).

- Binarize the digits (pixel > 8), fit for each pixel j (in raster order) a logistic regression of x_j on the 16 previous pixels, and sample 16 new binary digits pixel by pixel.

Exercise 26 (**Comparing generative models** $\star\star\star$)

A generator can be evaluated by a distance between the distribution of its samples and the one of held-out data. The *sliced Wasserstein distance* averages, over random directions, the 1D Wasserstein distance between the projected samples (computed by sorting); it detects both badly placed samples and missing modes. Copying the training data would also give a small distance: a generator must also be compared with the training set.

- Compute the sliced Wasserstein distance (200 directions) between 2000 fresh points of the two moons and the samples of the Gaussian, the mixture of Gaussians, the VAE, Real NVP, flow matching (100 steps) and the GAN, as well as the training set itself. Rank the generators.

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.